Informe de vulnerabilidad

Webhook de pago sin verificación de autenticidad — Sistema de Licencias de Pesca Deportiva

Sistema afectado	Sistema de Licencias de Pesca Deportiva (Laravel / Filament) — Ministerio de Ecología y Recursos Naturales Renovables. Sistema externo al monorepo **botonera-pagos**, desarrollado por otro equipo.
Componente	Integración con la pasarela de pago **Click de Pago (Macro)**: endpoint de notificación de pagos (webhook).
Tipo de vulnerabilidad	CWE-345 — Insufficient Verification of Data Authenticity. CWE-306 — Missing Authentication for Critical Function.
Severidad	**Crítica. Permite obtener un bien/documento oficial (licencia de pesca) sin pago real, de forma remota, sin autenticación y sin interacción del usuario.**
Estado	Confirmado por revisión de código fuente. **No explotado** contra el entorno de producción: este informe no incluye ni fue generado a partir de una explotación real.
Fecha	10 de noviembre de 2026
Reportado por	Equipo de desarrollo de la botonera de pagos, durante el relevamiento de pasarelas de pago para los trámites del Ministerio.

Nota de alcance: *este documento describe una vulnerabilidad encontrada por revisión de código en un sistema que no fue construido por este equipo y que es independiente de la botonera de pagos. Se documenta con fines de evaluación de riesgo y para fundamentar una decisión de reemplazo del sistema; no constituye, ni fue precedido por, una prueba de explotación en el entorno real.*

1. Resumen ejecutivo

El sistema de Licencias de Pesca Deportiva recibe notificaciones de pago del proveedor **Click de Pago** (del banco Macro) mediante un endpoint HTTP público (`POST /api/webhook/payment`). Este endpoint confía en el contenido del cuerpo de la petición para decidir si un pago fue realizado, sin verificar de ninguna forma que la notificación provenga realmente de Click de Pago.

Como consecuencia, cualquier persona que conozca o adivine el identificador de una licencia pendiente de pago puede enviar una petición HTTP falsa a ese endpoint, simulando un pago exitoso, y el sistema:

- marca la licencia como activa (`active = true`);
- genera el PDF de la licencia oficial, firmado con los logos del Ministerio y un código QR de validación;
- envía por correo electrónico la licencia al usuario, como si el pago hubiera sido legítimo.

Todo esto ocurre **sin que exista ningún pago real**, sin autenticación, sin firma criptográfica y sin ningún control de origen de la petición (el código no valida IP, token ni firma alguna). El impacto es la emisión fraudulenta de un documento oficial del Ministerio y la pérdida de los ingresos por esa licencia.

2. Flujo normal de pago (cómo debería funcionar)

1. El usuario inicia el pago de su licencia desde `PayLicence.php`, que prepara un formulario con los datos de la transacción (monto, identificador, URLs de retorno) cifrados con AES y firmados con un hash SHA-256, y lo envía por POST a Click de Pago en una ventana emergente.

2. El usuario paga en la página de Click de Pago (fuera del sistema del Ministerio).

3. Click de Pago notifica el resultado de dos maneras: (a) redirige la ventana emergente a una página de *callback* que le avisa a la ventana principal por `postMessage`, y (b) **en paralelo, por su cuenta, hace un POST al webhook del servidor** (`/api/webhook/payment`) con el resultado real de la transacción. Es esta segunda vía, servidor-a-servidor, la que activa la licencia y genera el PDF.

El problema es que el servidor del Ministerio no tiene manera de distinguir un POST legítimo de Click de Pago de un POST falso enviado por cualquier otra persona, porque el endpoint del paso 3 no valida nada.

3. Código afectado

3.1 Endpoint sin autenticación ni middleware

Archivo: `routes/api.php`

```
Route::post('/webhook/payment', [WebhookController::class, 'handlePaymentWebhook']);
Route::get('/webhook/preview', [WebhookController::class, 'previewHtml']);
```

La ruta no tiene ningún `middleware` de autenticación, de verificación de firma, ni de limitación de origen. Es un endpoint público, accesible por cualquiera en Internet que conozca la URL del sistema.

3.2 El controlador confía ciegamente en el cuerpo de la petición

Archivo: `app/Http/Controllers/WebhookController.php`, *líneas 22-63*

```
public function handlePaymentWebhook(Request $request)
{
    Log::info('Webhook: '. time() .' - ' . $request->getContent());

    $json_data = $request->getContent();
    $array_data = json_decode($json_data, true);

    $payment = Payment::where('id_transaction', $array_data['TransaccionComercioId'])
        ->update([
            'state' => $array_data['Estado'],
            'type_transaction' => $array_data['Tipo'],
            'json_data' => $json_data,
        ]);

    if($payment == null){
        Transaction::create([
            'id_transaction' => $array_data['TransaccionComercioId'],
            'state' => $array_data['Estado'],
            'type_transaction' => $array_data['Tipo'],
            'json_data' => $json_data,
        ]);
    }

    $id_licence = explode('_', $array_data['TransaccionComercioId'])[0];
    $licencia = Licence::with('user')->where('id', $id_licence)->first();

    if($array_data['Estado'] == 'REALIZADA'){

        $user = User::where('email', $licencia->user->email)->first();
        $pdfPath = $this->savePdf($array_data['TransaccionComercioId'], $licencia, $user);
        $relativePath = Str::after($pdfPath, 'storage/app/public/');

        Licence::where('id', $id_licence)->update([
```

```
            'active' => true,
            'pdf_path' => $relativePath,
        ]);

        $user->notify(new LicenseEmail($pdfPath));
    }

    return response()->json(['message' => 'OK']);
}
```

Observaciones puntuales sobre este código:

- **No hay verificación de firma ni de secreto compartido.** El manual de Click de Pago no documenta un mecanismo de firma para este webhook en particular, pero aun así el código no valida ningún token, cabecera ni origen propio.
- **No hay control de IP de origen.** Click de Pago publica un listado cerrado de IPs públicas desde las que se originan sus notificaciones (ver sección 6); el código no las compara con `$request->ip()`.
- **El estado se toma literalmente del cuerpo recibido.** La condición `if ($array_data['Estado'] == 'REALIZADA')` decide, por sí sola, si se activa la licencia y se emite el PDF. Quien controla el cuerpo del POST controla el resultado.
- **El identificador de la licencia se deriva del mismo campo no verificado.** `$id_licence = explode('_', $array_data['TransaccionComercioId'])[0]` — el atacante elige directamente qué licencia activar, simplemente eligiendo el valor de `TransaccionComercioId`.
- **Nunca se vuelve a consultar a Click de Pago** para confirmar que la transacción exista y esté realmente pagada (lo que sí sería posible: la propia API de Click de Pago ofrece `GET /transaction/search` y `GET /transactions` para esto).
- **Siempre responde 200 OK**, lo cual en sí mismo es correcto (evita reintentos del proveedor), pero combinado con todo lo anterior significa que un ataque también recibe una confirmación de éxito.

3.3 El identificador de transacción es predecible

Archivo: `app/Filament/App/Resources/LicenceResource/Pages/PayLicence.php`, *línea 557*

```
$transaccionId = $this->licence_id.'_Time:'.time();
```

El identificador de transacción tiene el formato `<id_licencia>_Time:<timestamp_unix>`. El `id_licencia` es autoincremental y, en la práctica, público o fácil de inferir: aparece en la URL de pago de cada usuario (`/app/licences/{record}`), y el webhook solo usa la parte anterior al primer guion bajo (`explode('_', ...)[0]`). Esto significa que ni siquiera es necesario adivinar el timestamp: con solo el número de licencia alcanza para construir un `TransaccionComercioId` que el webhook acepte.

4. Escenario de explotación (conceptual, no ejecutado)

A continuación se describe, a nivel conceptual, cómo se encadenan los puntos anteriores para obtener una licencia sin pagar. Esta secuencia no fue ejecutada contra el sistema real; se presenta únicamente para que quede clara la explotabilidad del problema.

1. Un usuario inicia normalmente la compra de una licencia (queda creada con `state = 'CREADA'`) y nota el número identificador de su licencia (por ejemplo, visible en la URL de la pantalla de pago).
2. En lugar de completar el pago, construye manualmente una petición HTTP `POST` hacia el endpoint público del webhook, imitando el formato que usa Click de Pago, con el `TransaccionComercioId` igual al número de su licencia.

3. El servidor recibe esa petición, no valida su origen ni autenticidad, interpreta `Estado = "REALIZADA"` como un pago exitoso, marca la licencia como activa, genera el PDF oficial y lo envía por correo.
4. El usuario descarga o recibe por correo una licencia de pesca deportiva válida —con el QR de verificación funcionando— sin haber pagado.

Forma de la petición falsificada (ilustrativa)

A modo ilustrativo, y sin indicar el dominio real del sistema, la petición tendría esta forma (basada únicamente en los campos que el código en 3.2 lee del JSON):

```
POST /api/webhook/payment HTTP/1.1
Host: <sistema-de-pesca>
Content-Type: application/json

{
  "TransaccionComercioId": "<numero_de_licencia>",
  "Tipo": "PAGO",
  "Estado": "REALIZADA"
}
```

No se requiere autenticación, token, ni conocimiento de ningún secreto: los tres campos leídos por `handlePaymentWebhook` son exactamente estos tres.

5. Impacto

Dimensión	Impacto
Económico	Emisión de licencias de pesca deportiva sin que el Ministerio perciba el pago correspondiente. El impacto escala con la cantidad de licencias que se emitan así; no requiere privilegios ni conocimientos técnicos avanzados.
Integridad de registros	Se generan registros de pago (`Payment`, `Transaction`) con estado "REALIZADA" que no corresponden a una transacción real en Click de Pago, contaminando la conciliación contable.
Documento oficial	El PDF emitido es indistinguible de una licencia legítima (mismos logos, formato y código QR de validación), lo que además puede dificultar los controles en el campo por parte de guardafaunas o autoridades de control.
Explotabilidad	**Alta.** No requiere autenticación, no requiere conocer ningún secreto criptográfico, y el identificador necesario es predecible o directamente visible para cualquier usuario registrado.
Alcance	Cualquier licencia en estado "CREADA" (pago iniciado pero no confirmado) es susceptible, incluidas las de terceros, en la medida en que se conozca o se infiera su número.

6. Causa raíz

El diseño del endpoint parte de un supuesto incorrecto: que nadie más que Click de Pago va a conocer la URL del webhook ni el formato de su cuerpo. "Seguridad por oscuridad" no es una defensa: la URL (`/api/webhook/payment`) y el formato del JSON están documentados públicamente en el manual de

integración de Click de Pago y son, en la práctica, deducibles por cualquiera con conocimientos básicos de HTTP.

Faltan, como mínimo, dos de las siguientes verificaciones (lo ideal es combinar varias):

- **Verificación de origen por IP.** Click de Pago publica, en su manual de integración, el listado cerrado de rangos de IP desde donde se originan sus notificaciones (10 rangos /27, /28 y /29). El código no los compara.
- **Reconfirmación activa contra la API.** Ante cada notificación, volver a consultar el estado real de la transacción contra la propia API de Click de Pago (`GET /transaction/search` o `GET /transactions`, con el token de sesión del comercio) y solo confiar en esa respuesta, nunca en el cuerpo recibido.
- **Secreto compartido o firma propia.** Acordar con Click de Pago (o agregar del lado propio) un token o firma en el webhook que permita rechazar peticiones que no la incluyan.
- **Identificador de transacción no predecible.** Usar un identificador aleatorio (UUID) en lugar de `<id_licencia>_Time:<timestamp>`, para que no alcance con conocer el número de licencia.

7. Recomendación

Corto plazo (si el sistema sigue en uso): implementar al menos la verificación de IP de origen y la reconfirmación activa contra la API de Click de Pago antes de activar cualquier licencia. Es un cambio acotado al `WebhookController` y no requiere rediseñar el resto del sistema.

Mediano plazo: este hallazgo, junto con el hecho de que el sistema es externo al resto de los trámites digitalizados del Ministerio (no comparte base, paneles, ni el patrón de validación de webhooks que se está definiendo para las nuevas integraciones de pago), respalda la decisión de incorporar la licencia de pesca a la plataforma unificada de trámites y pagos, donde la verificación de webhooks (reconfirmación contra la API del proveedor antes de dar por pagada cualquier orden) es un requisito de diseño desde el inicio.

8. Metodología de este informe

Este hallazgo se obtuvo exclusivamente por **revisión manual del código fuente** del sistema de Licencias de Pesca (archivos PHP/Laravel compartidos para evaluar su integración con Click de Pago), sin ejecutar ninguna petición contra el entorno de producción ni contra ningún otro entorno del sistema. No se generó, activó ni emitió ninguna licencia real como parte de esta evaluación.

Archivos revisados: `app/Http/Controllers/WebhookController.php`, `app/Filament/App/Resources/LicenceResource/Pages/PayLicence.php`, `routes/api.php`, `routes/web.php`, `app/Models/Payment.php`, `database/seeders/PaymentConfigurationSeeder.php`.